Vector Embedding for Capability Composition

Sidharth V. Jain, B.Tech.
Department of Computer Science & Engineering
Semester 5
Roll No. 62

PCCST503 Machine Learning — Assignment 2

October 4, 2026

Abstract

This project develops a structured vector representation for formally specified states, goals, and executable capabilities. The design follows the central requirement of the assignment: the representation must preserve the information needed for capability identity, compatibility, and composition rather than treating generic semantic similarity as equivalent to executability.

An e-commerce purchase workflow is used as the experimental application. The implementation represents capabilities such as `CreateOrder`, `MakePayment`, and `SendNotification`; constructs their valid composition; and distinguishes the valid sequence `CreateOrder` → `MakePayment` from the invalid sequence `CreateOrder` → `CancelCart`. Operational attributes including execution time, resource cost, monetary cost, risk, reliability, and availability are represented explicitly.

The final prototype provides state and goal encoders, structured capability embeddings, a formal compatibility function, capability composition, goal relevance, automated evaluation, plots, and unit tests.

1 Problem Definition

The assignment asks for a vector representation of formally specified capabilities and for an investigation of whether that representation can preserve the relationships required for composition. The problem is therefore not only one of semantic similarity. A capability must also be executable in the context produced by a preceding capability.

A capability is represented by the formal tuple

$$C = (T, I, O, P, E, K, R, Q, Rel, A, M), \tag{1}$$

where the components represent its type, inputs, outputs, preconditions, effects, constraints, resources, operational properties, reliability, availability, and execution mechanism.

The experimental application is an e-commerce purchase workflow. Its terminal goal is

$$G = \{\texttt{Order.exists} = \texttt{true}, \texttt{Payment.status} = \texttt{SUCCESS}, \texttt{Notification.sent} = \texttt{true}\}. \tag{2}$$

The intended capability chain is

$$\texttt{CreateOrder} \rightarrow \texttt{MakePayment} \rightarrow \texttt{SendNotification}. \tag{3}$$

2 Design Requirements

The implementation is designed to support the following requirements from the assignment:

1. representation of capability identity;
2. representation of states and goals;
3. explicit input/output representation;
4. explicit precondition/effect representation;
5. constraints and resource representation;
6. operational attributes such as cost and reliability;
7. distinction between similarity and composability;
8. composition of compatible capabilities;
9. goal-relevance analysis; and
10. quantitative experimental evaluation.

The design deliberately separates vector similarity from formal compatibility. This distinction is central to the implementation.

3 Proposed Representation

3.1 State and Goal Representation

States and goals are formal key–value structures. A common symbolic vector space is used for both, so the representation does not depend on a particular sequence of operations.

For a state or goal object X, a feature is created for each key–value pair:

$$f_{k,v}(X) = \begin{cases} 1 & \text{if } X[k] = v, \\ 0 & \text{otherwise.} \end{cases} \tag{4}$$

This gives a deterministic representation that is appropriate for the small formal application used in the prototype.

3.2 Capability Representation

The capability embedding is factorized into five blocks:

$$\phi(C) = [v_{\text{id}}; v_{\text{io}}; v_{\text{logic}}; v_{\text{resource}}; v_{\text{operation}}]. \tag{5}$$

The blocks encode:

- identity and execution mechanism;
- inputs and outputs;
- preconditions, effects, and constraints;
- resources; and
- operational properties.

Each block is vectorized and standardized independently before concatenation. This prevents numerical quantities such as execution time from dominating symbolic execution structure.

Table 1: Capability embedding blocks and weights.

Block	Weight
Identity / mechanism	0.10
Inputs / outputs	0.25
Preconditions / effects / constraints	0.40
Resources	0.10
Operational attributes	0.15

The final capability vector has dimension **70** for the experimental dataset.

4 Similarity Model

Similarity is calculated using weighted block-level cosine similarity:

$$Sim(C_i, C_j) = \sum_{b \in B} w_b \cos\left(v_i^{(b)}, v_j^{(b)}\right), \tag{6}$$

where B is the set of five blocks and $\sum_b w_b = 1$.

The logic block receives the largest weight because preconditions and effects are especially important for executable capability composition.

Similarity is used to express functional or structural relatedness. It is not used as the sole criterion for deciding whether a composition is valid.

5 Compatibility Model

For two capabilities A and B, the implementation defines:

$$Comp(A, B) = 0.55M_{EP} + 0.25M_{OI} + 0.20M_K, \tag{7}$$

where:

- M_{EP} is the effect–precondition match;
- M_{OI} is the output–input match; and
- M_K is constraint compatibility.

A composition-ready pair is accepted at a threshold of

$$Comp(A, B) \geq 0.75. \tag{8}$$

For example, `CreateOrder` establishes `Order.exists=true` and `Order.status=CREATED`, which satisfies the formal requirements of `MakePayment`. In contrast, `CancelCart` requires an absent order, creating a direct conflict with the effects of `CreateOrder`.

6 Capability Composition

For a valid sequence

$$C_1 \to C_2 \to \cdots \to C_n, \tag{9}$$

the implementation constructs a composite capability

$$C_{1:n} = C_n \circ \cdots \circ C_2 \circ C_1. \tag{10}$$

The composite:

- inherits the external preconditions of the first capability;
- accumulates effects in execution order;
- retains required external inputs;
- exposes the outputs of the final capability;
- combines constraints and resources;
- sums execution time, resource cost, and monetary cost;
- accumulates the modeled risk; and
- multiplies reliability.

For the evaluated sequence `CreateOrder` → `MakePayment` → `SendNotification`, the reliability is

$$0.99 \times 0.98 \times 0.97 = 0.941094. \tag{11}$$

The total execution time is

$$100 + 300 + 150 = 550 \text{ ms}. \tag{12}$$

The resulting composite also establishes all three terminal goal facts.

7 Implementation

The implementation is written in Python using NumPy and scikit-learn.

The main interfaces are:

```
StateGoalEncoder.encode_state(state)
StateGoalEncoder.encode_goal(goal)
CapabilityEncoder.encode(capability)
CapabilityEncoder.similarity(a, b)
compatibility(a, b)
compose([c1, c2, ...])
goal_relevance(capability, goal)
```

The package contains:

- `dataset.json`: formal application and capabilities;
- `capability_embedding.py`: implementation;
- `run_experiments.py`: experiments and figures;
- `test_capability_embedding.py`: automated tests;
- `artifacts/`: numerical results and figures; and
- `technical_report.md`: report-ready evidence narrative.

8 Experimental Methodology

Four experimental groups were used.

8.1 Compatibility Classification

Eight formally selected pairs were evaluated: four compatible and four incompatible. The decision threshold was 0.75.

8.2 Alternative Implementations

`CreateOrder` was compared with a database implementation and a GUI implementation. These capabilities have different mechanisms but share important functional structure.

8.3 Irrelevant Capability

`CheckInventory` was compared with `CreateOrder`. This tests whether a different capability is distinguished from alternative implementations of the same functional operation.

8.4 Operational Attributes

Execution time, resource cost, monetary cost, risk, reliability, and availability are retained explicitly. They are used for a separate operational analysis rather than being conflated with formal compatibility.

9 Results

9.1 Compatibility

The compatibility classifier achieved:

Table 2: Compatibility classification results.

Metric	Value
True positives	4
True negatives	4
False positives	0
False negatives	0
Accuracy	1.00
Precision	1.00
Recall	1.00
F1	1.00

The perfect score should be interpreted as performance on the controlled synthetic evaluation set, not as evidence of generalization to arbitrary applications.

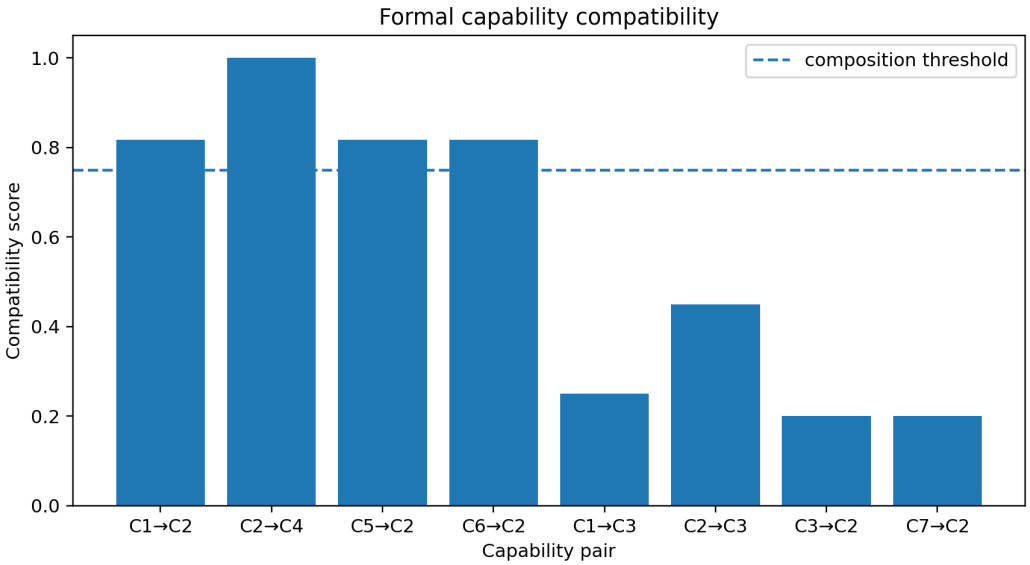


Figure 1: Formal compatibility scores for the evaluated capability pairs. The dashed line is the composition threshold.

9.2 Alternative Implementations and Irrelevance

The structured similarity results were:

Table 3: Similarity results.

Pair	Structured similarity
`C1` ∼ `C5`	0.361
`C1` ∼ `C6`	0.262
`C1` ∼ `C7`	-0.35

The average similarity of the two alternative implementations is approximately 0.311, while the irrelevant comparison is approximately -0.353. This provides the expected directional separation in this controlled representation.

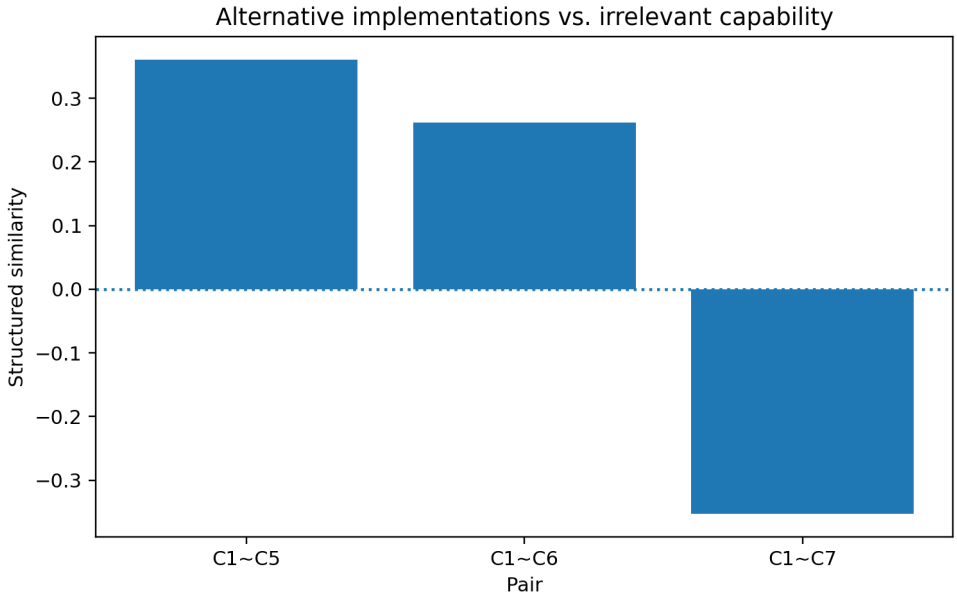


Figure 2: Structured similarity for alternative implementations and an irrelevant capability.

9.3 Goal Relevance

Goal relevance is calculated as the fraction of terminal goal facts directly established by a capability's effects. The generated results are available in `artifacts/goal_relevance.csv`.

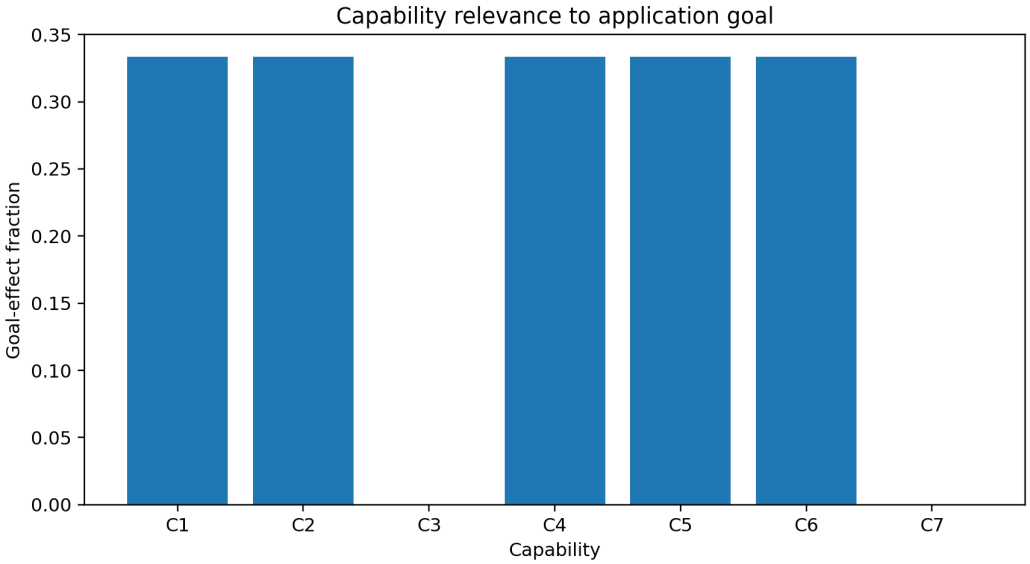


Figure 3: Capability relevance to the formal application goal.

9.4 Composite Operational Properties

The final composite has:

Table 4: Operational properties of C1∘C2∘C4.

Property	Value
Execution time	550 ms
Resource cost	7
Monetary cost	0.23
Accumulated risk	0.40
Reliability	0.941094
Availability	1

10 Analysis

The experiments support the central design decision that similarity and composability should be represented separately.

Vector similarity is useful for functional and structural relatedness. This is particularly important when alternative implementations have different mechanisms but similar inputs, outputs, preconditions, and effects.

Formal compatibility is required for executable composition. A capability may be related to another capability while still failing to satisfy its preconditions or accept its outputs. The explicit compatibility function therefore provides information that cosine similarity alone cannot provide.

The composite representation also retains operational properties. This allows the system to distinguish the formal fact that a composition is valid from the practical question of how expensive or reliable that composition is.

11 Limitations

The implementation is a controlled prototype.

1. The application dataset is synthetic and small.
2. Constraint reasoning covers the formal patterns represented in the dataset rather than arbitrary constraint languages.
3. Embedding weights are designed rather than learned from a large corpus of application traces.
4. The state and goal encoders are deterministic symbolic encoders.
5. The operational utility analysis is an illustrative selection heuristic and is separate from formal compatibility.
6. The compatibility results are evaluated on a small, explicitly constructed test set.

Accordingly, the results demonstrate the feasibility and structure of the representation rather than broad empirical generalization.

12 Conclusion

This project implements a structured vector representation for formally specified capabilities and separates two relationships that are easily conflated: vector similarity and executable compatibility.

The representation combines capability identity, inputs, outputs, preconditions, effects, constraints, resources, and operational properties. A formal compatibility function then evaluates whether one capability can follow another, while the composition operator constructs a composite capability and propagates its effects and operational properties.

The resulting prototype supports state encoding, goal encoding, capability encoding, similarity, compatibility, composition, goal relevance, automated evaluation, and reproducible tests. It therefore provides a complete submission implementation for the assignment's capability-embedding objective.

Submission Note

The accompanying code and generated artifacts are the authoritative implementation evidence. All numerical values reported here are generated from the included experimental scripts and stored under `artifacts/`. The evaluation should be interpreted in the context of the synthetic application and the formal test cases used by the prototype.